TECNOLÓGICO DE ESTUDIOS SUPERIORES DE ECATEPEC

DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

PROGRAMACIÓN DE BASE DE DATOS

TAREA 4: TECNOLOGÍAS PARA CONECTIVIDAD A BASES DE DATOS: ADO.NET

PROFESORA:

ALEJANDRA GONZALEZ SÁNCHEZ

NOMBRE:

MONTERRUBIO LÓPEZ LUIS GAEL

GRUPO: 5701

FECHA: 30-10-26

1. Introducción y objetivo

Este documento explica las características de ADO.NET, su arquitectura, sus ventajas y desventajas, y su uso en un módulo funcional en C# que realiza operaciones CRUD sobre una base de datos PostgreSQL.

El módulo forma parte de la plataforma IoT Analytics Dashboard de NexoDev Solutions: administra los registros de sensores de la planta (temperatura, humedad, presión, energía, vibración y mantenimiento). Se ejecuta con Docker y se desplegó en un clúster de Kubernetes, igual que el resto del ecosistema de la empresa.

Repositorio del proyecto: https://github.com/luisgael-ml/iot-analytics-adonet

Competencia específica: conectar una base de datos utilizando un lenguaje de programación actualizado (clave de actividad BDM-1301T01CE01A07).

Objetivos específicos:

1. Investigar y documentar las características de ADO.NET y su uso con PostgreSQL.
2. Desarrollar un módulo en C# que implemente operaciones CRUD con ADO.NET y PostgreSQL.
3. Configurar un entorno con Docker y Kubernetes para probar la conectividad y el funcionamiento del módulo.
4. Documentar el diseño del módulo y los resultados obtenidos.
5. Presentar la integración del módulo en el proyecto IoT Analytics Dashboard.

2. Marco teórico: ADO.NET

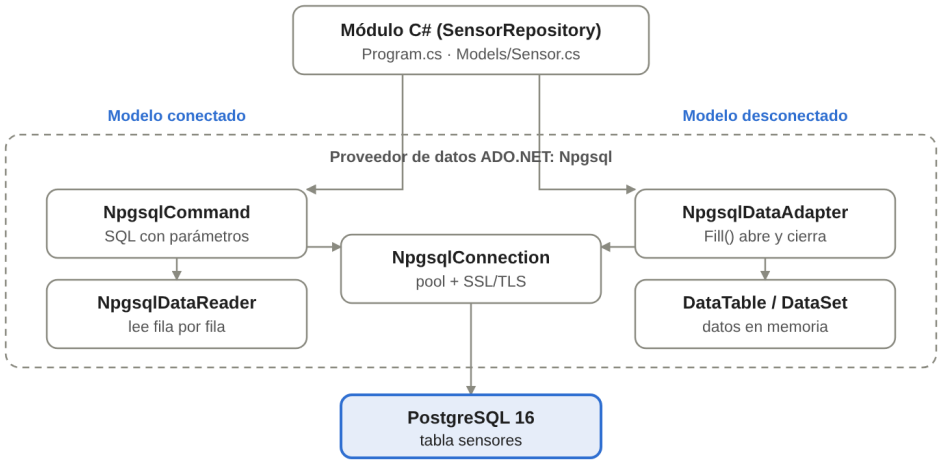
2.1 ¿Qué es ADO.NET?

ADO.NET (ActiveX Data Objects para .NET) es el conjunto de clases de .NET para acceder a bases de datos y otras fuentes de datos. Forma parte de la biblioteca base de .NET, en el espacio de nombres `System.Data`.

ADO.NET define clases abstractas comunes (`DbConnection`, `DbCommand`, `DbDataReader`, `DbDataAdapter`) y cada gestor de base de datos las implementa con un **proveedor de datos**. Para PostgreSQL, el proveedor es **Npgsql**, que se instala como paquete NuGet.

2.2 Arquitectura

ADO.NET tiene dos partes: el **proveedor de datos**, que se comunica con la base de datos, y el **DataSet**, que guarda datos en memoria sin necesidad de una conexión abierta. Con ellas se trabaja en dos modelos: conectado y desconectado.



Arquitectura de ADO.NET con el proveedor Npgsql y sus dos modelos de acceso.

MODELO	CLASES	CÓMO FUNCIONA	CUÁNDO CONVIENE
Conectado	Connection, Command, DataReader	La conexión permanece abierta mientras se leen las filas, una por una y solo hacia adelante	Lecturas rápidas y grandes volúmenes; operaciones INSERT, UPDATE y DELETE
Desconectado	DataAdapter, DataSet, DataTable	El DataAdapter abre la conexión, copia los datos a memoria y la cierra	Trabajar con los datos sin conexión, enlazarlos a interfaces o procesarlos varias veces

2.3 Componentes principales

COMPONENTE	CLASE EN NPGSQL	FUNCIÓN
Connection	`NpgsqlConnection`	Abre la sesión con la base de datos; se configura con una cadena de conexión
Command	`NpgsqlCommand`	Ejecuta una sentencia SQL o un procedimiento almacenado
Parameter	`NpgsqlParameter`	Valor que se envía separado del SQL (@nombre); evita la inyección SQL
DataReader	`NpgsqlDataReader`	Lee los resultados fila por fila, solo hacia adelante (modelo conectado)
DataAdapter	`NpgsqlDataAdapter`	Puente entre la base y un DataSet; llena tablas en memoria con Fill()
DataSet / DataTable	`System.Data`	Tablas en memoria, independientes del gestor de base de datos
Transaction	`NpgsqlTransaction`	Agrupa varias operaciones: se confirman todas (Commit) o ninguna (Rollback)

Métodos de ejecución de `NpgsqlCommand`:

MÉTODO	DEVUELVE	USO EN EL MÓDULO
`ExecuteReader()`	Un DataReader con las filas	Listar sensores
`ExecuteNonQuery()`	Número de filas afectadas	UPDATE y DELETE
`ExecuteScalar()`	El primer valor de la primera fila	INSERT ... RETURNING id

2.4 Proveedores de datos

GESTOR	PROVEEDOR ADO.NET	PAQUETE
SQL Server	SqlClient	Microsoft.Data.SqlClient
PostgreSQL	Npgsql (usado en este proyecto)	Npgsql
MySQL / MariaDB	MySqlConnector	MySqlConnector
Oracle	Oracle Data Provider (ODP.NET)	Oracle.ManagedDataAccess.Core
SQLite	Microsoft.Data.Sqlite	Microsoft.Data.Sqlite

2.5 Características

- **Independencia del gestor:** las clases base son las mismas; cambiar de base de datos implica cambiar el proveedor y la cadena de conexión.
- **Modelo conectado y desconectado:** permite elegir entre velocidad (DataReader) y trabajo en memoria (DataSet).
- **Pool de conexiones integrado:** las conexiones cerradas regresan a un pool y se reutilizan, sin abrir una nueva cada vez.
- **Consultas parametrizadas:** los valores viajan separados del SQL, lo que evita la inyección SQL.
- **Transacciones:** varias operaciones se confirman o se deshacen como una sola unidad.
- **Liberación de recursos con `using`:** la conexión y el comando se cierran automáticamente al terminar el bloque.

2.6 Ventajas y desventajas

VENTAJAS	DESVENTAJAS
Máximo rendimiento y control: el desarrollador escribe el SQL exacto	Más código repetitivo: abrir conexión, crear comando, asignar parámetros y leer columnas a mano
Parte estándar de .NET, sin dependencias pesadas	El SQL va en texto: los errores de sintaxis aparecen hasta la ejecución
Pool de conexiones y transacciones incluidos	Mapear filas a objetos (Sensor) se hace manualmente
Modelo desconectado para trabajar con datos en memoria	El DataSet consume más memoria que un DataReader en volúmenes grandes
Funciona con cualquier gestor que tenga proveedor	Algunas funciones de SQL son propias de cada gestor (por ejemplo, RETURNING en PostgreSQL)

2.7 ADO.NET frente a JDBC y Entity Framework

ASPECTO	ADO.NET (C#)	JDBC (JAVA, TAREA 3)	ENTITY FRAMEWORK CORE
Nivel	Bajo nivel, SQL manual	Bajo nivel, SQL manual	Alto nivel (ORM), genera el SQL
Conexión	NpgsqlConnection	Connection (DriverManager)	DbContext
Consulta	NpgsqlCommand + @parámetros	PreparedStatement + ?	Consultas LINQ
Resultados	DataReader o DataTable	ResultSet	Objetos de C#
Pool	Integrado en el proveedor	Requiere DataSource (HikariCP)	Usa el de ADO.NET

Entity Framework Core está construido sobre ADO.NET: internamente usa los mismos proveedores, como Npgsql.

3. Herramientas y requisitos

HERRAMIENTA	VERSIÓN	USO EN EL PROYECTO
Docker Desktop	Actual	Ejecutar PostgreSQL y el SDK de .NET en contenedores
Kubernetes (Docker Desktop, kubeadm)	1.34.1	Desplegar PostgreSQL y el módulo como en un entorno real
.NET SDK (imagen mcr.microsoft.com/dotnet/sdk)	8.0	Compilar y ejecutar el módulo en C#
Npgsql (proveedor ADO.NET)	8.0.5	Conectar C# con PostgreSQL
PostgreSQL (imagen Docker)	16	Gestor de base de datos
kubectl	Incluido en Docker Desktop	Administrar el clúster de Kubernetes
Git + GitHub	Actual	Control de versiones

No es necesario instalar .NET en la computadora: el SDK se ejecuta dentro de un contenedor.

4. Configuración del entorno

4.1 Proyecto y proveedor ADO.NET

El archivo `IotAnalytics.csproj` define una aplicación de consola en .NET 8 y agrega el proveedor Npgsql. `dotnet restore` lo descarga de NuGet al compilar.

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <RootNamespace>IotAnalytics</RootNamespace>
    <InvariantGlobalization>true</InvariantGlobalization>
  </PropertyGroup>

  <ItemGroup>
    <!-- Proveedor ADO.NET para PostgreSQL -->
    <PackageReference Include="Npgsql" Version="8.0.5" />
  </ItemGroup>

</Project>
```

4.2 Base de datos

El script `sql/init.sql` crea la tabla `sensores` con tres registros iniciales. Docker y Kubernetes lo ejecutan automáticamente la primera vez que arranca PostgreSQL.

```
-- Tabla de sensores del proyecto IoT Analytics Dashboard
CREATE TABLE IF NOT EXISTS sensores (
    id        SERIAL PRIMARY KEY,           -- identificador autoincremental
    nombre    VARCHAR(50)  NOT NULL,        -- nombre del sensor
```

```
    tipo      VARCHAR(30)  NOT NULL,        -- temperatura, humedad, presión...
    ubicacion VARCHAR(50),                  -- área de la planta
    valor     NUMERIC(10,2),                -- última lectura
    fecha     TIMESTAMP DEFAULT NOW()       -- fecha de registro
);

-- Datos iniciales
INSERT INTO sensores (nombre, tipo, ubicacion, valor) VALUES
('SEN-T01', 'Temperatura', 'Línea 1', 24.50),
('SEN-H01', 'Humedad', 'Almacén', 55.00),
('SEN-P01', 'Presión', 'Caldera', 101.30);
```

4.3 Docker Compose

`docker-compose.yml` levanta PostgreSQL con SSL y un contenedor con el SDK de .NET que compila y ejecuta el módulo. El proyecto tiene nombre propio y usa el puerto 5433 para no chocar con otros contenedores.

```
# Nombre propio del proyecto para no mezclarse con otros proyectos de Docker
name: iot-analytics-adonet

services:
  # Base de datos PostgreSQL 16 con SSL activado
  postgres:
    image: postgres:16
    container_name: pg-iot-analytics
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD:-admin123}   # del archivo .env; admin123 si no existe
      POSTGRES_DB: iotanalytics
    # Activa SSL con el certificado autofirmado que trae la imagen
    command: >
      -c ssl=on
      -c ssl_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
      -c ssl_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
    ports:
      - "5433:5432"     # 5433 en la PC para no chocar con otros PostgreSQL
    volumes:
      # Crea la tabla y los datos iniciales la primera vez que arranca
      - ./sql/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d iotanalytics"]
      interval: 3s
      retries: 10

  # Módulo C# (SDK de .NET 8 dentro del contenedor, no requiere instalarlo)
  app:
    image: mcr.microsoft.com/dotnet/sdk:8.0
    working_dir: /app
    environment:
      DB_HOST: postgres
      DB_PASSWORD: ${DB_PASSWORD:-admin123}
      DB_SSLMODE: Require
    volumes:
      - ./:/app
      - nuget:/root/.nuget/packages   # guarda Npgsql descargado para no bajarlo cada vez
    command: dotnet run
    stdin_open: true
    tty: true
    depends_on:
      postgres:
```

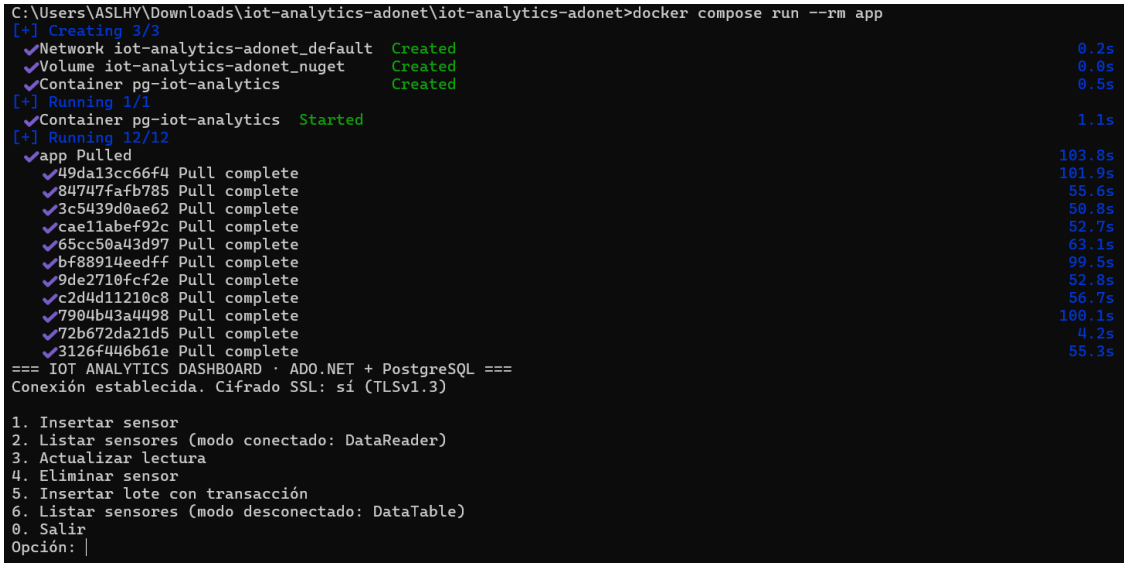
```
        condition: service_healthy

volumes:
  nuget:
```

Ejecución:

```
docker compose run --rm app
```

Evidencia 1: el módulo compila dentro del contenedor, se conecta a PostgreSQL con SSL (TLSv1.3) y muestra el menú.



Ejecución del módulo con Docker Compose.

4.4 Imagen del módulo para Kubernetes

El `Dockerfile` compila el módulo en dos etapas: la imagen del SDK compila y publica, y la imagen final solo contiene el runtime de .NET y el programa compilado.

```
# Imagen del módulo para Kubernetes (compilación en dos etapas)
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY IotAnalytics.csproj .
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /out --no-restore

FROM mcr.microsoft.com/dotnet/runtime:8.0
WORKDIR /app
COPY --from=build /out .
ENTRYPOINT ["dotnet", "IotAnalytics.dll"]

docker build -t iot-analytics-adonet:1.0 .
```

Evidencia 2: construcción de la imagen. `dotnet restore` descarga Npgsql y `dotnet publish` compila el módulo sin errores.

```
C:\Users\ASLHY\Downloads\iot-analytics-adonet\iot-analytics-adonet>docker build -t iot-analytics-adonet:1.0 .
[+] Building 98.6s (15/15) FINISHED                                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                 0.3s
 => => transferring dockerfile: 394B                                                                                                 0.1s
 => [internal] load metadata for mcr.microsoft.com/dotnet/sdk:8.0                                                                    0.5s
 => [internal] load metadata for mcr.microsoft.com/dotnet/runtime:8.0                                                                4.5s
 => [internal] load .dockerignore                                                                                                    0.5s
 => => transferring context: 72B                                                                                                     0.0s
 => [build 1/6] FROM mcr.microsoft.com/dotnet/sdk:8.0@sha256:6ba9644da07bb5b560a402acec9ca38711a0762969f1b61da261df5c9f6f85da      14.8s
 => => resolve mcr.microsoft.com/dotnet/sdk:8.0@sha256:6ba9644da07bb5b560a402acec9ca38711a0762969f1b61da261df5c9f6f85da              0.8s
 => [stage-1 1/3] FROM mcr.microsoft.com/dotnet/runtime:8.0@sha256:67e16b7d568208713e05db7f7bcc81827679a636222160bf57bb148acd23622e  6.0s
 => => resolve mcr.microsoft.com/dotnet/runtime:8.0@sha256:67e16b7d568208713e05db7f7bcc81827679a636222160bf57bb148acd23622e          1.1s
 => [internal] load build context                                                                                                    2.8s
 => => transferring context: 19.95kB                                                                                                 1.0s
 => [stage-1 2/3] WORKDIR /app                                                                                                       7.9s
 => [build 2/6] WORKDIR /src                                                                                                         1.5s
 => [build 3/6] COPY IotAnalytics.csproj .                                                                                           1.2s
 => [build 4/6] RUN dotnet restore                                                                                                  19.7s
 => [build 5/6] COPY . .                                                                                                             1.1s
 => [build 6/6] RUN dotnet publish -c Release -o /out --no-restore                                                                  30.3s
 => [stage-1 3/3] COPY --from=build /out .                                                                                           1.5s
 => exporting to image                                                                                                              11.7s
 => => exporting layers                                                                                                             10.0s
 => => exporting manifest sha256:1647ed98a4c86a97180449aab9d36d834e4108063922ad3e248e8b187639c6f6                                    0.2s
 => => exporting config sha256:67d72c59597ff44e55b23b4ca4d773d6e5958a649c9f6f3aa7955133d57880d8                                      0.2s
 => => exporting attestation manifest sha256:f04e20ef8fead44f40887b95c20ecf9e974a3311825c527829da2b97099fa9a0                        0.3s
 => => exporting manifest list sha256:a7f2eb2c69bc58b5647bc0142417d8185274fd879a13d4467894aa792b525010                               0.1s
 => => naming to docker.io/library/iot-analytics-adonet:1.0                                                                          0.0s
 => => unpacking to docker.io/library/iot-analytics-adonet:1.0                                                                       0.4s
```

Construcción de la imagen iot-analytics-adonet:1.0.

4.5 Clúster de Kubernetes

Se activó Kubernetes en Docker Desktop con el método kubeadm (un nodo, versión 1.34.1) y se seleccionó su contexto en kubectl.

```
kubectl config use-context docker-desktop
kubectl get nodes
```

Evidencia 3: el nodo `docker-desktop` está en estado *Ready*.

```
C:\Users\ASLHY\Downloads\iot-analytics-adonet\iot-analytics-adonet>kubectl config use-context docker-desktop
Switched to context "docker-desktop".

C:\Users\ASLHY\Downloads\iot-analytics-adonet\iot-analytics-adonet>kubectl get nodes
NAME             STATUS   ROLES           AGE    VERSION
docker-desktop   Ready    control-plane   119s   v1.34.1
```

Nodo del clúster de Kubernetes listo.

El archivo `k8s/iot-analytics.yaml` define cinco recursos:

RECURSO	NOMBRE	FUNCIÓN
Secret	db-secret	Guarda la contraseña de la base de datos fuera del código
ConfigMap	db-init	Contiene init.sql, que PostgreSQL ejecuta al iniciar
Deployment	postgres	Mantiene en ejecución el pod de PostgreSQL 16 con SSL
Service	postgres	Nombre de red estable (postgres:5432) para llegar a la base de datos
Job	iot-analytics-demo	Ejecuta el módulo C# una vez en modo demostración

```
# Despliegue del módulo IoT Analytics Dashboard en Kubernetes
# 1) Secret con la contraseña  2) ConfigMap con el script SQL
# 3) PostgreSQL (Deployment + Service)  4) Job que ejecuta el módulo C# en modo demo
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:
  DB_PASSWORD: admin123
---
apiVersion: v1
kind: ConfigMap
metadata:
```

```
  name: db-init
data:
  init.sql: |
    -- Tabla de sensores del proyecto IoT Analytics Dashboard
    CREATE TABLE IF NOT EXISTS sensores (
        id        SERIAL PRIMARY KEY,           -- identificador autoincremental
        nombre    VARCHAR(50)  NOT NULL,        -- nombre del sensor
        tipo      VARCHAR(30)  NOT NULL,        -- temperatura, humedad, presión...
        ubicacion VARCHAR(50),                  -- área de la planta
        valor     NUMERIC(10,2),                -- última lectura
        fecha     TIMESTAMP DEFAULT NOW()       -- fecha de registro
    );

    -- Datos iniciales
    INSERT INTO sensores (nombre, tipo, ubicacion, valor) VALUES
    ('SEN-T01', 'Temperatura', 'Línea 1', 24.50),
    ('SEN-H01', 'Humedad', 'Almacén', 55.00),
    ('SEN-P01', 'Presión', 'Caldera', 101.30);
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          args: ["-c", "ssl=on",
                 "-c", "ssl_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem",
                 "-c", "ssl_key_file=/etc/ssl/private/ssl-cert-snakeoil.key"]
          env:
            - name: POSTGRES_DB
              value: iotanalytics
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: DB_PASSWORD
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: init
              mountPath: /docker-entrypoint-initdb.d
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres", "-d", "iotanalytics"]
            initialDelaySeconds: 5
            periodSeconds: 3
      volumes:
        - name: init
          configMap:
```

```
            name: db-init
---
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
---
apiVersion: batch/v1
kind: Job
metadata:
  name: iot-analytics-demo
spec:
  backoffLimit: 3
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: iot-analytics
          image: iot-analytics-adonet:1.0
          imagePullPolicy: Never      # usa la imagen construida localmente con docker build
          args: ["--demo"]
          env:
            - name: DB_HOST
              value: postgres         # nombre del Service de PostgreSQL
            - name: DB_SSLMODE
              value: Require
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: DB_PASSWORD
```

5. Implementación del módulo

El módulo separa la conexión, el modelo de datos y las operaciones SQL en clases distintas. `SensorRepository` usa los dos modelos de ADO.NET: el conectado para el CRUD y el desconectado para consultar en memoria.

5.1 Estructura del proyecto

```
iot-analytics-adonet/
├── IotAnalytics.csproj        → proyecto .NET 8 y paquete Npgsql
├── Program.cs                 → menú interactivo y modo --demo
├── Data/Conexion.cs           → cadena de conexión, SSL y pool
├── Data/SensorRepository.cs   → CRUD con ADO.NET
├── Models/Sensor.cs           → modelo de datos
├── sql/init.sql               → tabla y datos iniciales
├── docker-compose.yml         → PostgreSQL + SDK de .NET
├── Dockerfile                 → imagen para Kubernetes
└── k8s/iot-analytics.yaml     → Secret, ConfigMap, PostgreSQL y Job
```

5.2 Conexion.cs

Arma la cadena de conexión con `NpgsqlConnectionStringBuilder`, sin concatenar texto. La contraseña se lee de `DB_PASSWORD`, la conexión exige SSL y el pool queda limitado a 20 conexiones. `EsperarDisponible()` reintenta la conexión mientras PostgreSQL arranca, algo necesario en Kubernetes.

```
using Npgsql;

namespace IotAnalytics.Data;

/// <summary>
/// Crea las conexiones ADO.NET (NpgsqlConnection) hacia PostgreSQL.
/// - Las credenciales se leen de variables de entorno (no están en el código).
/// - La conexión viaja cifrada con SSL/TLS.
/// - Npgsql administra un pool de conexiones automáticamente.
/// </summary>
public static class Conexion
{
    public static string CadenaConexion()
    {
        string? password = Environment.GetEnvironmentVariable("DB_PASSWORD");
        if (string.IsNullOrWhiteSpace(password))
            throw new InvalidOperationException("Falta la variable de entorno DB_PASSWORD");

        // El builder arma la cadena de conexión sin concatenar texto a mano
        var builder = new NpgsqlConnectionStringBuilder
        {
            Host = Environment.GetEnvironmentVariable("DB_HOST") ?? "localhost",
            Port = int.Parse(Environment.GetEnvironmentVariable("DB_PORT") ?? "5432"),
            Database = "iotanalytics",
            Username = Environment.GetEnvironmentVariable("DB_USER") ?? "postgres",
            Password = password,
            // Require: la conexión se rechaza si el servidor no acepta SSL
            SslMode = Enum.Parse<SslMode>(Environment.GetEnvironmentVariable("DB_SSLMODE") ?? "Require",
true),
            // Pool de conexiones: se reutilizan en lugar de abrir una nueva cada vez
            Pooling = true,
            MinPoolSize = 1,
            MaxPoolSize = 20
        };
        return builder.ConnectionString;
    }

    /// <summary>Devuelve una conexión nueva (cerrada). Se abre con Open().</summary>
    public static NpgsqlConnection Crear() => new NpgsqlConnection(CadenaConexion());

    /// <summary>
    /// Espera a que PostgreSQL acepte conexiones (útil en Docker y Kubernetes,
    /// donde la aplicación puede arrancar antes que la base de datos).
    /// </summary>
    public static bool EsperarDisponible(int intentos = 30)
    {
        for (int i = 1; i <= intentos; i++)
        {
            try
            {
                using var con = Crear();
                con.Open();
```

```
                return true;
            }
            catch (Exception e) when (i < intentos)
            {
                Console.WriteLine($"Esperando a PostgreSQL ({i}/{intentos}): {e.Message}");
                Thread.Sleep(2000);
            }
            catch (Exception e)
            {
                Console.WriteLine($"No se pudo conectar: {e.Message}");
            }
        }
        return false;
    }
}
```

5.3 Sensor.cs

Representa una fila de la tabla `sensores`.

```
namespace IotAnalytics.Models;

/// <summary>
/// Modelo de datos: cada objeto equivale a una fila de la tabla sensores.
/// </summary>
public class Sensor
{
    public int Id { get; set; }
    public string Nombre { get; set; } = "";
    public string Tipo { get; set; } = "";
    public string? Ubicacion { get; set; }
    public decimal? Valor { get; set; }
    public DateTime Fecha { get; set; }

    public Sensor() { }

    public Sensor(string nombre, string tipo, string? ubicacion, decimal valor)
    {
        Nombre = nombre;
        Tipo = tipo;
        Ubicacion = ubicacion;
        Valor = valor;
    }

    public override string ToString() =>
        $"{Id,-4} {Nombre,-10} {Tipo,-12} {Ubicacion,-12} {Valor,8:0.00}  {Fecha:yyyy-MM-dd HH:mm:ss}";
}
```

5.4 SensorRepository.cs

Contiene el CRUD. Cada método abre una conexión dentro de un bloque `using`, que la devuelve al pool al terminar. Todos los valores se envían como parámetros.

```
using System.Data;
using IotAnalytics.Models;
using Npgsql;

namespace IotAnalytics.Data;

/// <summary>
/// Repositorio: concentra todas las operaciones SQL sobre la tabla sensores
```

```
/// usando los dos modelos de ADO.NET:
///   - Conectado:    NpgsqlConnection + NpgsqlCommand + NpgsqlDataReader
///   - Desconectado: NpgsqlDataAdapter + DataTable
/// </summary>
public class SensorRepository
{
    // ---------- MODELO CONECTADO ----------

    /// <summary>CREATE: inserta un sensor y devuelve el id generado.</summary>
    public int Insertar(Sensor s)
    {
        const string sql = @"INSERT INTO sensores (nombre, tipo, ubicacion, valor)
                             VALUES (@nombre, @tipo, @ubicacion, @valor)
                             RETURNING id";
        using var con = Conexion.Crear();
        con.Open();
        using var cmd = new NpgsqlCommand(sql, con);
        // Parámetros: evitan la inyección SQL
        cmd.Parameters.AddWithValue("nombre", s.Nombre);
        cmd.Parameters.AddWithValue("tipo", s.Tipo);
        cmd.Parameters.AddWithValue("ubicacion", (object?)s.Ubicacion ?? DBNull.Value);
        cmd.Parameters.AddWithValue("valor", (object?)s.Valor ?? DBNull.Value);
        // ExecuteScalar devuelve la primera columna de la primera fila (el id)
        return Convert.ToInt32(cmd.ExecuteScalar());
    }

    /// <summary>READ: devuelve todos los sensores leyendo fila por fila con un DataReader.</summary>
    public List<Sensor> Listar()
    {
        const string sql = "SELECT id, nombre, tipo, ubicacion, valor, fecha FROM sensores ORDER BY id";
        var lista = new List<Sensor>();
        using var con = Conexion.Crear();
        con.Open();
        using var cmd = new NpgsqlCommand(sql, con);
        using NpgsqlDataReader reader = cmd.ExecuteReader();
        while (reader.Read())                       // avanza una fila (solo hacia adelante)
        {
            lista.Add(new Sensor
            {
                Id = reader.GetInt32(0),
                Nombre = reader.GetString(1),
                Tipo = reader.GetString(2),
                Ubicacion = reader.IsDBNull(3) ? null : reader.GetString(3),
                Valor = reader.IsDBNull(4) ? (decimal?)null : reader.GetDecimal(4),
                Fecha = reader.GetDateTime(5)
            });
        }
        return lista;
    }

    /// <summary>UPDATE: actualiza la lectura de un sensor. Devuelve true si existía.</summary>
    public bool ActualizarValor(int id, decimal nuevoValor)
    {
        const string sql = "UPDATE sensores SET valor = @valor, fecha = NOW() WHERE id = @id";
        using var con = Conexion.Crear();
        con.Open();
        using var cmd = new NpgsqlCommand(sql, con);
        cmd.Parameters.AddWithValue("valor", nuevoValor);
        cmd.Parameters.AddWithValue("id", id);
        return cmd.ExecuteNonQuery() > 0;           // filas afectadas
```

```
    }

    /// <summary>DELETE: elimina un sensor. Devuelve true si existía.</summary>
    public bool Eliminar(int id)
    {
        const string sql = "DELETE FROM sensores WHERE id = @id";
        using var con = Conexion.Crear();
        con.Open();
        using var cmd = new NpgsqlCommand(sql, con);
        cmd.Parameters.AddWithValue("id", id);
        return cmd.ExecuteNonQuery() > 0;
    }

    /// <summary>
    /// Inserta varios sensores dentro de una transacción:
    /// se guardan todos o ninguno (Commit / Rollback).
    /// </summary>
    public int InsertarLote(IEnumerable<Sensor> sensores)
    {
        const string sql = @"INSERT INTO sensores (nombre, tipo, ubicacion, valor)
                             VALUES (@nombre, @tipo, @ubicacion, @valor)";
        using var con = Conexion.Crear();
        con.Open();
        using NpgsqlTransaction tx = con.BeginTransaction();
        try
        {
            int total = 0;
            foreach (var s in sensores)
            {
                using var cmd = new NpgsqlCommand(sql, con, tx);
                cmd.Parameters.AddWithValue("nombre", s.Nombre);
                cmd.Parameters.AddWithValue("tipo", s.Tipo);
                cmd.Parameters.AddWithValue("ubicacion", (object?)s.Ubicacion ?? DBNull.Value);
                cmd.Parameters.AddWithValue("valor", (object?)s.Valor ?? DBNull.Value);
                total += cmd.ExecuteNonQuery();
            }
            tx.Commit();                            // confirma todo el lote
            return total;
        }
        catch
        {
            tx.Rollback();                          // deshace todo si algo falla
            throw;
        }
    }

    /// <summary>Consulta al servidor si la conexión actual va cifrada.</summary>
    public string EstadoSSL()
    {
        const string sql = "SELECT ssl, version FROM pg_stat_ssl WHERE pid = pg_backend_pid()";
        using var con = Conexion.Crear();
        con.Open();
        using var cmd = new NpgsqlCommand(sql, con);
        using var reader = cmd.ExecuteReader();
        return reader.Read() && reader.GetBoolean(0) ? $"sí ({reader.GetString(1)})" : "no";
    }

    // ---------- MODELO DESCONECTADO ----------

    /// <summary>
```

```
    /// READ desconectado: el DataAdapter abre la conexión, llena un DataTable
    /// en memoria y la cierra. Los datos se pueden recorrer sin conexión abierta.
    /// </summary>
    public DataTable ListarDesconectado()
    {
        const string sql = "SELECT id, nombre, tipo, ubicacion, valor FROM sensores ORDER BY id";
        using var con = Conexion.Crear();
        using var adapter = new NpgsqlDataAdapter(sql, con);
        var tabla = new DataTable("sensores");
        adapter.Fill(tabla);                        // abre, consulta y cierra la conexión
        return tabla;
    }
}
```

5.5 Program.cs

Muestra el estado de la conexión y un menú con las seis operaciones. Con el argumento `--demo` ejecuta todas las operaciones sin interacción; así corre dentro de Kubernetes.

```
using System.Data;
using System.Globalization;
using IotAnalytics.Data;
using IotAnalytics.Models;

// Punto de entrada del módulo IoT Analytics Dashboard (ADO.NET + PostgreSQL).
//   dotnet run            -> menú interactivo
//   dotnet run -- --demo  -> ejecuta todas las operaciones automáticamente (usado en Kubernetes)

Console.OutputEncoding = System.Text.Encoding.UTF8;
var repo = new SensorRepository();
var inv = CultureInfo.InvariantCulture;   // acepta "26.80" con punto decimal

Console.WriteLine("=== IOT ANALYTICS DASHBOARD · ADO.NET + PostgreSQL ===");
if (Conexion.EsperarDisponible())
{
    try { Console.WriteLine($"Conexión establecida. Cifrado SSL: {repo.EstadoSSL()}"); }
    catch (Exception e) { Console.WriteLine($"Error al verificar SSL: {e.Message}"); }
}

if (args.Contains("--demo"))
{
    Demo();
    return;
}

int opcion;
do
{
    Console.WriteLine();
    Console.WriteLine("1. Insertar sensor");
    Console.WriteLine("2. Listar sensores (modo conectado: DataReader)");
    Console.WriteLine("3. Actualizar lectura");
    Console.WriteLine("4. Eliminar sensor");
    Console.WriteLine("5. Insertar lote con transacción");
    Console.WriteLine("6. Listar sensores (modo desconectado: DataTable)");
    Console.WriteLine("0. Salir");
    Console.Write("Opción: ");
    if (!int.TryParse(Console.ReadLine(), out opcion)) { Console.WriteLine("Opción inválida."); opcion =
-1; continue; }
```

```
    try
    {
        switch (opcion)
        {
            case 1:
                string nombre = Leer("Nombre: ");
                string tipo = Leer("Tipo: ");
                string ubicacion = Leer("Ubicación: ");
                decimal valor = decimal.Parse(Leer("Valor: "), inv);
                int id = repo.Insertar(new Sensor(nombre, tipo, ubicacion, valor));
                Console.WriteLine($"Sensor insertado con id {id}.");
                break;
            case 2:
                Imprimir(repo.Listar());
                break;
            case 3:
                int idAct = int.Parse(Leer("ID del sensor: "));
                decimal nuevo = decimal.Parse(Leer("Nuevo valor: "), inv);
                Console.WriteLine(repo.ActualizarValor(idAct, nuevo) ? "Lectura actualizada." : "ID no
encontrado.");
                break;
            case 4:
                int idDel = int.Parse(Leer("ID del sensor: "));
                Console.WriteLine(repo.Eliminar(idDel) ? "Sensor eliminado." : "ID no encontrado.");
                break;
            case 5:
                int n = repo.InsertarLote(LoteEjemplo());
                Console.WriteLine($"Transacción confirmada: {n} sensores insertados.");
                break;
            case 6:
                ImprimirTabla(repo.ListarDesconectado());
                break;
            case 0:
                Console.WriteLine("Hasta luego.");
                break;
            default:
                Console.WriteLine("Opción inválida.");
                break;
        }
    }
    catch (FormatException)
    {
        Console.WriteLine("Dato con formato incorrecto.");
    }
    catch (Exception e)
    {
        // Errores de conexión o de SQL se muestran sin cerrar el programa
        Console.WriteLine($"Error: {e.Message}");
    }
} while (opcion != 0);

// ---------- Modo demostración (sin interacción) ----------
void Demo()
{
    Paso("P2 · READ con DataReader");
    Imprimir(repo.Listar());

    Paso("P3 · CREATE con ExecuteScalar (RETURNING id)");
    int nuevoId = repo.Insertar(new Sensor("SEN-E01", "Energía", "Línea 2", 12.75m));
    Console.WriteLine($"Sensor insertado con id {nuevoId}.");
```

```
    Paso("P4 · UPDATE con ExecuteNonQuery");
    Console.WriteLine(repo.ActualizarValor(nuevoId, 15.40m) ? $"Lectura del id {nuevoId} actualizada a
15.40." : "ID no encontrado.");

    Paso("P5 · Transacción (Commit de un lote)");
    Console.WriteLine($"Transacción confirmada: {repo.InsertarLote(LoteEjemplo())} sensores
insertados.");

    Paso("P6 · DELETE con ExecuteNonQuery");
    Console.WriteLine(repo.Eliminar(nuevoId) ? $"Sensor {nuevoId} eliminado." : "ID no encontrado.");

    Paso("P7 · Id inexistente");
    Console.WriteLine(repo.ActualizarValor(9999, 1m) ? "Lectura actualizada." : "ID 9999 no
encontrado.");

    Paso("P8 · READ desconectado con DataAdapter + DataTable");
    ImprimirTabla(repo.ListarDesconectado());

    Console.WriteLine();
    Console.WriteLine("Demostración terminada correctamente.");
}

// ---------- Utilidades ----------
static string Leer(string etiqueta)
{
    Console.Write(etiqueta);
    return Console.ReadLine() ?? "";
}

static void Paso(string titulo)
{
    Console.WriteLine();
    Console.WriteLine($"--- {titulo} ---");
}

static List<Sensor> LoteEjemplo() => new()
{
    new Sensor("SEN-M01", "Mantenimiento", "Caldera", 1m),
    new Sensor("SEN-V01", "Vibración", "Prensa 3", 4.20m)
};

static void Imprimir(List<Sensor> sensores)
{
    Console.WriteLine($"{"ID",-4} {"NOMBRE",-10} {"TIPO",-12} {"UBICACIÓN",-12} {"VALOR",8}  FECHA");
    sensores.ForEach(s => Console.WriteLine(s));
}

static void ImprimirTabla(DataTable tabla)
{
    Console.WriteLine($"DataTable '{tabla.TableName}': {tabla.Rows.Count} filas en memoria");
    Console.WriteLine($"{"ID",-4} {"NOMBRE",-10} {"TIPO",-14} {"UBICACIÓN",-12} {"VALOR",8}");
    foreach (DataRow fila in tabla.Rows)
    {
        Console.WriteLine($"{fila["id"],-4} {fila["nombre"],-10} {fila["tipo"],-14} {fila["ubicacion"],-
12} {fila["valor"],8}");
    }
}
```

5.6 Seguridad de la conexión

MEDIDA	CÓMO SE APLICA	QUÉ PROTEGE
Cifrado SSL/TLS	PostgreSQL arranca con ssl=on y Npgsql se conecta con SslMode=Require	Los datos y la contraseña viajan cifrados
Credenciales fuera del código	DB_PASSWORD como variable de entorno; Secret en Kubernetes	La contraseña no queda en el código ni en el repositorio
Consultas parametrizadas	Parámetros @nombre, @id con NpgsqlParameter	Evita la inyección SQL
Transacciones	BeginTransaction, Commit y Rollback	Un lote nunca queda guardado a medias

6. Pruebas funcionales en Kubernetes

El módulo se ejecutó como Job dentro del clúster. Primero esperó a que PostgreSQL estuviera listo y después realizó todas las operaciones.

```
kubectl apply -f k8s/iot-analytics.yaml
kubectl wait --for=condition=complete job/iot-analytics-demo --timeout=300s
kubectl logs job/iot-analytics-demo
```

NO.	OPERACIÓN ADO.NET	RESULTADO ESPERADO	RESULTADO OBTENIDO
P1	Conexión con SSL (NpgsqlConnection)	Conexión cifrada	Correcto: reintentó 5 veces mientras PostgreSQL arrancaba y se conectó con TLSv1.3
P2	READ con DataReader	Lista los 3 sensores iniciales	Correcto: SEN-T01, SEN-H01 y SEN-P01
P3	CREATE con ExecuteScalar	Inserta y devuelve el id	Correcto: sensor SEN-E01 insertado con id 4
P4	UPDATE con ExecuteNonQuery	Actualiza la lectura	Correcto: id 4 actualizado a 15.40
P5	Transacción (Commit)	Inserta un lote completo	Correcto: 2 sensores insertados en una transacción
P6	DELETE con ExecuteNonQuery	Elimina el sensor	Correcto: sensor 4 eliminado
P7	Id inexistente	Informa que no existe	Correcto: "ID 9999 no encontrado"
P8	READ con DataAdapter + DataTable	Llena una tabla en memoria	Correcto: DataTable con 5 filas

Evidencia 4: registro del Job con la conexión y las pruebas P1 a P5.

```
C:\Users\ASLHY\Downloads\iot-analytics-adonet\iot-analytics-adonet>kubectl logs job/iot-analytics-demo
=== IOT ANALYTICS DASHBOARD · ADO.NET + PostgreSQL ===
Esperando a PostgreSQL (1/30): Failed to connect to 10.101.80.101:5432
Esperando a PostgreSQL (2/30): Failed to connect to 10.101.80.101:5432
Esperando a PostgreSQL (3/30): Failed to connect to 10.101.80.101:5432
Esperando a PostgreSQL (4/30): Failed to connect to 10.101.80.101:5432
Esperando a PostgreSQL (5/30): Failed to connect to 10.101.80.101:5432
Conexión establecida. Cifrado SSL: sí (TLSv1.3)

--- P2 · READ con DataReader ---
ID   NOMBRE     TIPO         UBICACIÓN       VALOR  FECHA
1    SEN-T01    Temperatura  Línea 1         24.50  2026-10-30 07:17:47
2    SEN-H01    Humedad      Almacén         55.00  2026-10-30 07:17:47
3    SEN-P01    Presión      Caldera        101.30  2026-10-30 07:17:47

--- P3 · CREATE con ExecuteScalar (RETURNING id) ---
Sensor insertado con id 4.

--- P4 · UPDATE con ExecuteNonQuery ---
Lectura del id 4 actualizada a 15.40.

--- P5 · Transacción (Commit de un lote) ---
Transacción confirmada: 2 sensores insertados.
```

Conexión con SSL, lectura, inserción, actualización y transacción.

Evidencia 5: pruebas P6 a P8 y el resultado final en el DataTable.

```
--- P6 · DELETE con ExecuteNonQuery ---
Sensor 4 eliminado.

--- P7 · Id inexistente ---
ID 9999 no encontrado.

--- P8 · READ desconectado con DataAdapter + DataTable ---
DataTable 'sensores': 5 filas en memoria
ID   NOMBRE     TIPO           UBICACIÓN       VALOR
1    SEN-T01    Temperatura    Línea 1         24.50
2    SEN-H01    Humedad        Almacén         55.00
3    SEN-P01    Presión        Caldera        101.30
5    SEN-M01    Mantenimiento  Caldera          1.00
6    SEN-V01    Vibración      Prensa 3         4.20

Demostración terminada correctamente.
```

Eliminación, id inexistente y lectura en modo desconectado.

Evidencia 6: recursos del clúster. El pod de PostgreSQL sigue en ejecución, el Service tiene su IP interna y el Job terminó (Complete 1/1) en 21 segundos.

```
C:\Users\ASLHY\Downloads\iot-analytics-adonet\iot-analytics-adonet>kubectl get pods,svc,jobs
NAME                             READY   STATUS      RESTARTS   AGE
pod/iot-analytics-demo-rzh7c     0/1     Completed   0          92s
pod/postgres-55675db787-gtz4h    1/1     Running     0          93s

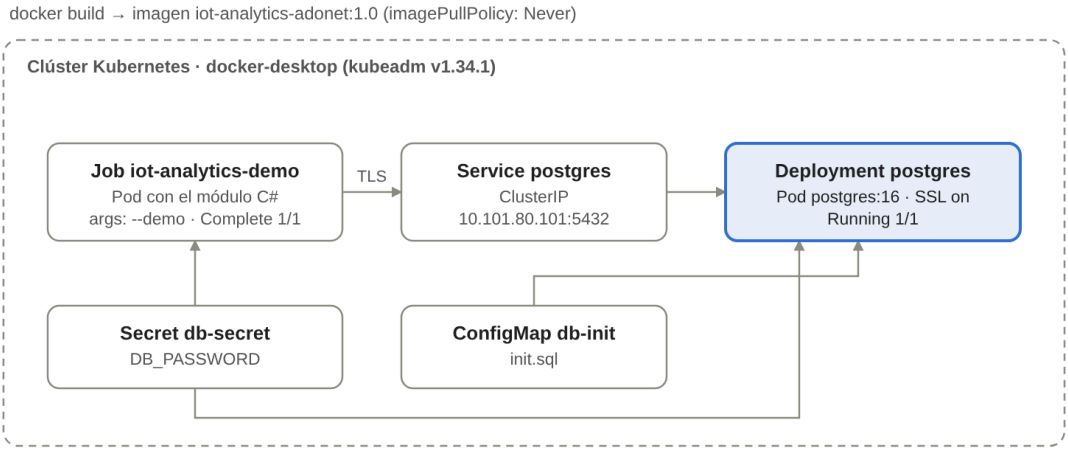
NAME                 TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP    3m52s
service/postgres     ClusterIP   10.101.80.101   <none>        5432/TCP   93s

NAME                           STATUS     COMPLETIONS   DURATION   AGE
job.batch/iot-analytics-demo   Complete   1/1           21s        92s
```

Pods, Service y Job en Kubernetes.

7. Presentación: integración en IoT Analytics Dashboard

El módulo se integra al ecosistema de NexoDev Solutions como un servicio más dentro de Kubernetes, el mismo stack que usa la empresa.



Despliegue del módulo en el clúster de Kubernetes.

1. **Construcción:** el código se compila en una imagen de Docker (iot-analytics-adonet:1.0).
2. **Configuración:** la contraseña llega desde un Secret y el esquema de la base desde un ConfigMap, sin modificar el código.
3. **Conexión:** el módulo localiza la base por el nombre del Service (postgres), no por una IP fija, y se conecta con SSL.
4. **Ejecución:** el Job ejecuta las operaciones con ADO.NET y termina; el registro queda disponible con kubectl logs.
5. **Compatibilidad:** los datos usan la misma tabla de sensores que la Tarea 3 (Java + JDBC), así que ambos módulos pueden trabajar sobre la misma base.

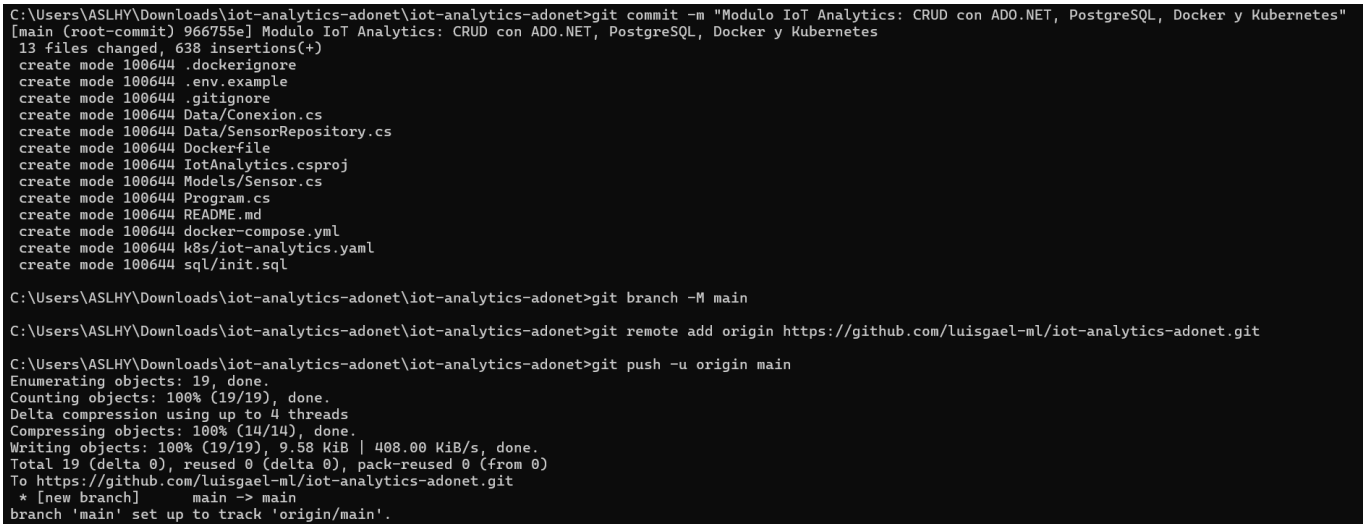
Para llevarlo a producción, el Job se convertiría en un Deployment que consuma lecturas desde RabbitMQ y las guarde por lotes con transacciones, con métricas en Prometheus.

8. Control de versiones y recursos compartidos

La actividad se desarrolló de forma individual. El proyecto se publicó en un repositorio público: https://github.com/luisgael-ml/iot-analytics-adonet

```
git init
git add .
git commit -m "Modulo IoT Analytics: CRUD con ADO.NET, PostgreSQL, Docker y Kubernetes"
git branch -M main
git remote add origin https://github.com/luisgael-ml/iot-analytics-adonet.git
git push -u origin main
```

Evidencia 7: el commit se registró a nombre de Luis Gael Monterrubio López y se publicó en la rama `main`.



Commit y publicación del módulo en GitHub.

8.1 Reflexión sobre la colaboración

Preparé el proyecto para que otros puedan aprovecharlo. El repositorio incluye un README con instrucciones, y el módulo se ejecuta con un solo comando de Docker o se despliega en Kubernetes sin instalar .NET. Un compañero puede revisar una implementación completa de ADO.NET y comparar los dos modelos de acceso con un ejemplo funcional.

También comparé ADO.NET con JDBC (sección 2.7) usando la misma base de datos de la Tarea 3. Esa comparación sirve como material de estudio para quien conozca una tecnología y quiera aprender la otra.

9. Conclusiones

ADO.NET permitió conectar un módulo en C# con PostgreSQL mediante el proveedor Npgsql, con control total sobre el SQL y un pool de conexiones integrado.

El modelo conectado (DataReader) resultó adecuado para el CRUD y las lecturas rápidas, mientras que el desconectado (DataAdapter y DataTable) permitió trabajar con los datos en memoria sin mantener la conexión abierta. Las transacciones garantizaron que un lote se guardara completo o no se guardara.

Docker y Kubernetes reprodujeron un entorno real: el módulo encontró la base por el nombre del Service, recibió la contraseña desde un Secret y se conectó con SSL, sin cambiar una línea de código respecto a la ejecución local.

Referencias

Microsoft. (s.f.). *ADO.NET overview*. https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ado-net-overview

Microsoft. (s.f.). *System.Data.Common namespace*. https://learn.microsoft.com/en-us/dotnet/api/system.data.common

Npgsql. (s.f.). *Npgsql documentation*. https://www.npgsql.org/doc/

The Kubernetes Authors. (s.f.). *Jobs*. https://kubernetes.io/docs/concepts/workloads/controllers/job/

Docker Inc. (s.f.). *Kubernetes on Docker Desktop*.
https://docs.docker.com/desktop/features/kubernetes/